



C# -OOP I

- class
- object
- Access Specifiers
- Fields
- Properties
- get, set
- Read only properties
- Constructors
- Destructors
- Object Initializer
- Methods
- Optional Parameters
- Named Parameters
- Positional Parameters
- Using params
- Local functions
- Project References
- Indexers



- A class is a user-defined blueprint or prototype from which objects are created. Basically, a class combines the fields and methods(member function which defines actions) into a single unit.

syntax:

```
[access-modifier] class identifier [:base-class]
{
  class-body
}
```

- **Modifiers:** A class can be public or internal etc. By default modifier of the class is *internal*.
- **Keyword class:** A *class* keyword is used to declare the type class.
- **Class Identifier:** The identifier(or name of the class) should begin with an initial letter which should be capitalized by convention.
- **Base class or Super class:** The name of the class's parent (superclass), if any, preceded by the : (*colon*). This is optional.
- **Body:** The class body is surrounded by { } (curly braces).

Class Name



**Properties of
a Class**



```
public class Student
{
    public string Name { get; set; }
    public int RollNumber { get; set; }

    public Student()
    {
    }
}
```



Constructor

- An object is an instance of a class.
- It is a basic unit of Object-Oriented Programming and represents real-life entities.

An object consists of :

- **State:** It is represented by attributes of an object. It also reflects the properties of an object.
- **Behavior:** It is represented by the methods of an object. It also reflects the response of an object with other objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.
- **Creating an Object of a class**

```
ClassName obj = new ClassName(); //syntax
```

```
Student st= new Student()
```

- public: The type or member can be accessed by any other code in the same assembly or another assembly that references it.
- private: The type or member can be accessed only by code in the same class or struct.
- protected: The type or member can be accessed only by code in the same class, or in a class that is derived from that class.
- internal: The type or member can be accessed by any code in the same assembly, but not from another assembly. In other words, internal types or members can be accessed from code that is part of the same compilation.
- protected internal: The type or member can be accessed by any code in the assembly in which it's declared, or from within a derived class in another assembly.
- private protected: The type or member can be accessed by types derived from the class that are declared within its containing assembly.

Summary table

Caller's location	public	protected internal	protected	internal	private protected	private
Within the class	✓	✓	✓	✓	✓	✓
Derived class (same assembly)	✓	✓	✓	✓	✓	✗
Non-derived class (same assembly)	✓	✓	✗	✓	✗	✗
Derived class (different assembly)	✓	✓	✓	✗	✗	✗
Non-derived class (different assembly)	✓	✗	✗	✗	✗	✗

- A class can have one or more fields.
- It is a class-level variable that holds a value.
- A class or struct may have instance fields, static fields, or both. Instance fields are specific to an instance of a type.
- Fields should not be accessed directly but always through Properties.
- The default access modifier is private.

```
class Student
{
    public int id;
}
```


- They are also known as class static fields, static class variables or static member data.
- These are useful when you want to define a value that applies to all objects of a given type.
- Often used to keep a count of the number of objects that have been instantiated.
- Often used to store a value that must be shared among all the instances.

```
public class Animal
{
    private int _Age;
    private static int _Count;
}
```

- You can access static variables using class name.
e.g. `Animal._Count`

- Properties are the special type of class members that provides a flexible mechanism to read, write, or compute the value of a private field.
- Properties can be used as if they are public data members, but they are actually special methods called **accessors**.
- This enables data to be accessed easily and helps to promote the flexibility and safety of methods.
- Encapsulation and hiding of information can also be achieved using properties.
- It uses pre-defined methods which are “get” and “set” methods which help to access and modify the properties.
- **Accessors:** The block of “set” and “get” is known as “Accessors”. It is very essential to restrict the accessibility of property.
- There are two type of accessors i.e. **get accessors** and **set accessors**.

- **There are different types of properties based on the “get” and “set” accessors:**
 1. **Read and Write Properties:** When property contains both get and set methods.
 2. **Read-Only Properties:** When property contains only get method.
 3. **Write Only Properties:** When property contains only set method.
 4. **Auto Implemented Properties:** When there is no additional logic in the property accessors and it introduce in C# 3.0.

■ The syntax for Defining Properties:

```
<access_modifier> <return_type> <property_name>
{
    get { // body }
    set { // body }
}
```

- Where, <access_modifier> can be public, private, protected or internal. Access modifiers define how users of the class can access the property.
- <return_type> can be any valid C# type.
- <property_name> can be user-defined.

```
class Student
{
    private int id;
    0 references
    public int StudentId
    {
        get { return id; }
        set { id = value; }
    }
}
```

- From C# 3.0 onwards, property declaration has been made easy if you don't want to apply some logic in getter or setter.
- Using auto-implemented property, you don't need to declare an underlying private field.
- C# compiler will automatically create it in IL code.

```
class Student
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

- You can initialize auto-implemented properties similarly to fields:

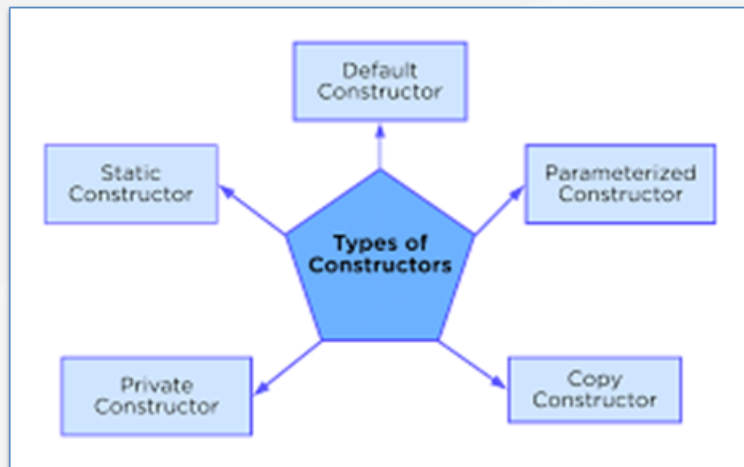
0 references

```
public string FirstName { get; set; } = "Jane";
```

- A constructor is a special method of the class which gets automatically invoked whenever an instance of the class is created.
- Like methods, a constructor also contains the collection of instructions that are executed at the time of Object creation.
- It is used to assign initial values to the **data members** of the same class.

```
class Student
{
    public Student()
    {
        //constructor
    }
}
```

- Constructor of a class must have the same name as the class name in which it resides.
- A constructor can not be abstract, final, and Synchronized.
- Within a class, you can create only one static constructor.
- A constructor doesn't have any return type, not even void.
- A static constructor cannot be a parameterized constructor.
- A class can have any number of constructors.
- Access modifiers can be used in constructor declaration to control its access i.e which other class can call the constructor.
- A class can have multiple constructors with different parameters but can only have one parameter less constructor.
- If no constructor is defined, the C# compiler would create it internally.




```
class Student
{
    private int id;
    private string name;
    private string email;
    private string phone;

    //Default Constructor
    public Student()
    {
        this.id = 1;
        this.name = string.Empty;
        this.email = string.Empty;
        this.phone = string.Empty;
    }

    //Parameterized Constructor
    public Student(int id, string name, string email, string phone)
    {
        this.id = id;
        this.name = name;
        this.email = email;
        this.phone = phone;
    }

    //Copy Constructor
    public Student(Student s)
    {
        this.id = s.id;
        this.name = s.name;
        this.email = s.email;
        this.phone = s.phone;
    }
}
```

■ Private Constructor

- If a constructor is created with private specifier is known as Private Constructor. It is not possible for other classes to derive from this class and also it's not possible to create an instance of this class.

■ Points To Remember :

- It is the implementation of a singleton class pattern.
- use private constructor when we have only static members.
- Using private constructor, prevents the creation of the instances of that class.

```
// declare private Constructor  
private MyClass()  
{  
}
```

- Static Constructor has to be invoked only once in the class and it has been invoked during the creation of the first reference to a static member in the class. A static constructor is initialized static fields or data of the class and to be executed only once.
- **Points To Remember :**
- It can't be called directly.
- When it is executing then the user has no control.
- It does not take access modifiers or any parameters.
- It is called automatically to initialize the class before the first instance created.

```
public class MyClass
{
    // declare static Constructor
    static MyClass()
    {
        // The following statement produces the first line of output,
        // and the line occurs only once.
        Console.WriteLine("Static Constructor");
    }
}
```

- Destructors in C# are methods inside the class used to destroy instances of that class when they are no longer needed. The Destructor is called implicitly by the .NET Framework's Garbage collector and therefore programmer has no control as when to invoke the destructor. An instance variable or an object is eligible for destruction when it is no longer reachable.
- **Important Points:**
 - A Destructor is unique to its class i.e. there cannot be more than one destructor in a class.
 - A Destructor has no return type and has exactly the same name as the class name (Including the same case).
 - It is distinguished apart from a constructor because of the Tilde symbol (~) prefixed to its name.
 - A Destructor does not accept any parameters and modifiers.
 - It cannot be defined in Structures. It is only used with classes.
 - It cannot be overloaded or inherited.
 - It is called when the program exits.
 - Internally, Destructor called the Finalize method on the base class of object.

```
class Example
{
    // Rest of the class
    // members and methods.
    // Destructor
    ~Example()
    {
        // Your code
    }
}
```

- C# 3.0 (.NET 3.5) introduced *Object Initializer Syntax*, a new way to initialize an object of a class or collection.
- Object initializers allow you to assign values to the fields or properties at the time of creating an object without invoking a constructor.

```
public class Student
{
    public int StudentID { get; set; }
    public string StudentName { get; set; }
    public int Age { get; set; }
    public string Address { get; set; }
}

class Program
{
    static void Main(string[] args)
    {
        Student std = new Student() { StudentID = 1,
                                       StudentName = "Bill",
                                       Age = 20,
                                       Address = "New York"
                                   };
    }
}
```

```
IList<Student> studentList = new List<Student>() {  
    new Student() { StudentID = 1, StudentName = "John" } ,  
    new Student() { StudentID = 2, StudentName = "Steve" } ,  
    new Student() { StudentID = 3, StudentName = "Bill" } ,  
    new Student() { StudentID = 3, StudentName = "Bill" } ,  
    new Student() { StudentID = 4, StudentName = "Ram" } ,  
    new Student() { StudentID = 5, StudentName = "Ron" }  
};
```

Advantages of Initializers

- Initializer syntax makes a code more readable, easy to add elements into the collection.
- Useful in multi-threading.

- A method can contain one or more statements to be executed as a single unit.
- A method may or may not return a value. A method can have one or more input parameters.

Syntax

```
[access-modifier] return-type MethodName(type parameterName1,  
type parameterName2,...) { }
```

Example:

```
public int Sum(int num1, int num2)  
{  
    var total = num1 + num2;  
    return total;  
}
```


- By default, all parameters of a method are required. But in C# 4.0, the concept of optional parameters was introduced that allows developers to declare parameters as optional.
- That means, if these arguments are not passed, they will be omitted from the execution. Optional parameters are not mandatory.

```
public int AddNumber(int firstNumber, int secondNumber = 0)
{
    return firstNumber + secondNumber;
}
```

```
public void MainMethod()
{
    AddNumber(12);
}
```

- Named Parameters allow developers to pass a method arguments with parameter names.
Prior to these this feature, the method parameters were passed using a sequence only. Now, using named parameters in C#, we can put any parameter in any sequence as long as the name is there.
- The right parameter value based on their names will be mapped to the right variable. The parameters name must match with the method definition parameter names.
- The syntax of parameter names is (parametername: value).

```
public static int AddNumber(int firstNumber, int secondNumber)
{
    return firstNumber + secondNumber;
}
```

```
AddNumber(firstNumber: 123, secondNumber: 64);
```

- C# supports the use of named arguments, in which you can change the order of arguments passed to a function, by prefixing an argument with the corresponding parameter name.
- An argument that is not prefixed with the name of a parameter is considered a positional argument.
- Positional arguments must come before named arguments and match the order of the corresponding parameters.

// Sample method with 3 parameters

```
public void Bark(int numTimes, string sound, double volume)
```

Examples of using both named and positional arguments.

// Example 1: All arguments are positional

```
myDog.Bark(2, "Woof", 10.0);
```

// Example 2: Only 1st argument is positional

```
myDog.Bark(2, volume: 10.0, sound: "Woof");
```

- It is useful when the programmer doesn't have any prior knowledge about the number of parameters to be used.
- By using params you are allowed to pass any variable number of arguments.
- Only one params keyword is allowed and no additional Params will be allowed in function declaration after a params keyword.
- The length of params will be zero if no arguments will be passed.

```
class Program
{
    // User defined function
    public void Show(params int[] val) // Params Parameter
    {
        for (int i = 0; i < val.Length; i++)
        {
            Console.WriteLine(val[i]);
        }
    }
    // Main function, execution entry point of the program
    static void Main(string[] args)
    {
        Program program = new Program(); // Creating Object
        program.Show(2, 4, 6, 8, 10, 12, 14); // Passing arguments of variable length
    }
}
```

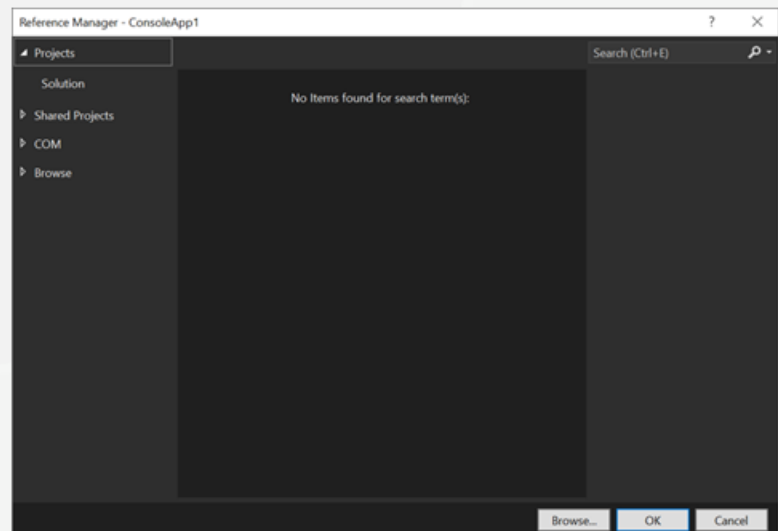
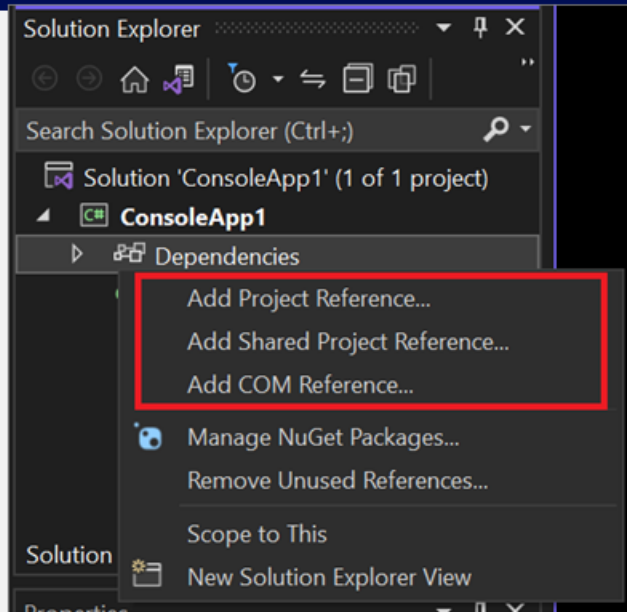
- The local function feature is introduced in **C# 7.0**.
- It allows you to declare a method inside the body of an already defined method.
- Or in other words, we can say that a local function is a private function of a function whose scope is limited to that function in which it is created.
- The type of local function is similar to the type of function in which it is defined.
- You can only call the local function from their container members.

- Local functions are declared inside methods, constructors, property accessors, event accessors, anonymous methods, lambda expressions, finalizers, and other local functions.
- You cannot declare a local function in the expression-bodied member.
- Local function makes your program more readable and also save you from mistakenly call method because you cannot call a local function directly.
- In local function, you are allowed to use *async* and *unsafe* modifiers.
- Local function can access the local variables that are defined inside the container method including method parameters.
- You are not allowed to use any member access modifiers in the local function definition, including private keyword because they are by default private and you are not allowed to make them public.
- You are also not allowed to use static keyword with local function.
- You are also not allowed to apply attributes to the local function, or to its parameters, or to its parameter type.
- Multiple local functions are allowed.
- Overloading is not allowed for local functions.

```
public class Program
{
    // Main method
    public static void Main()
    {
        // Local Function
        void AddValue(int a, int b)
        {
            Console.WriteLine("Value of a is: " + a);
            Console.WriteLine("Value of b is: " + b);
            Console.WriteLine("Sum of a and b is: {0}", a + b);
            Console.WriteLine();
        }

        // Calling Local function
        AddValue(20, 40);
        AddValue(40, 60);
    }
}
```

- A project reference is **a link from the current Studio project to another project.**
- The reference makes certain resources in the referenced project become available for use in the current project.
- **How to Add a reference**
 1. In **Solution Explorer**, right-click the **References** or **Dependencies** node, and then choose either **Add Project Reference** from the context menu. (You can right-click the project node and select **Add** from the fly-out menu to choose from these options, too.)
 2. **Reference Manager** opens and lists the available references by group.
 3. Select a reference to add, and then select **OK**.



- An indexer is a special type of property that allows a class or a structure to be accessed like an array for its internal collection.
- C# allows us to define custom indexers, generic indexers, and also overload indexers.
- An indexer can be defined the same way as property with this keyword and square brackets [].

```
[access_modifier] [return_type] this [argument_list]
{
    get
    {
        // get block code
    }
    set
    {
        // set block code
    }
}
```

- **access_modifier:** It can be public, private, protected or internal.
- **return_type:** It can be any valid C# type.
- **this:** It is the keyword which points to the object of the current class.
- **argument_list:** This specifies the parameter list of the indexer.
- **get{ } and set { }:** These are the accessors.

```
class StringDataStore
{
    private string[] strArr = new string[10]; // internal data storage
    public string this[int index]
    {
        get
        {
            if (index < 0 || index >= strArr.Length)
                throw new IndexOutOfRangeException("Index out of range");
            return strArr[index];
        }

        set
        {
            if (index < 0 || index >= strArr.Length)
                throw new IndexOutOfRangeException("Index out of range");
            strArr[index] = value;
        }
    }
}
```

```
// Main method
public static void Main()
{
    StringDataStore strStore = new StringDataStore();

    strStore[0] = "One";
    strStore[1] = "Two";
    strStore[2] = "Three";
    strStore[3] = "Four";

    for (int i = 0; i < 10; i++)
        Console.WriteLine(strStore[i]);
}
```

String Type Indexer

```
// string type indexer
public string this[string name]
{
    get
    {
        foreach (string str in strArr){
            if(str.ToLower() == name.ToLower())
                return str;
        }

        return null;
    }
}
```

```
static void Main(string[] args)
{
    StringDataStore strStore = new StringDataStore();

    strStore[0] = "One";
    strStore[1] = "Two";
    strStore[2] = "Three";
    strStore[3] = "Four";

    Console.WriteLine(strStore["one"]);
    Console.WriteLine(strStore["two"]);
    Console.WriteLine(strStore["Three"]);
    Console.WriteLine(strStore["Four"]);
}
```

- There are two types of Indexers i.e. One Dimensional Indexer & MultiDimensional Indexer. The above discussed is One Dimensional Indexer.
- Indexers can be overloaded.
- These are different from Properties.
- This enables the object to be indexed in a similar way to arrays.
- A set accessor will always assign the value while the get accessor will return the value.
- “this” keyword is always used to declare an indexer.
- To define the value being assigned by the set indexer, “value” keyword is used.
- Indexers are also known as the Smart Arrays or Parameterized Property in C#.
- Indexer can't be a static member as it is an instance member of the class.

Difference between Indexers and Properties

Indexers

Indexers are created with this keyword.

Indexers are identified by signature.

Indexers are accessed using indexes.

Indexer are instance member, so can't be static.

A get accessor of an indexer has the same formal parameter list as the indexer.

A set accessor of an indexer has the same formal parameter list as the indexer, in addition to the value parameter.

Properties

Properties don't require this keyword.

Properties are identified by their names.

Properties are accessed by their names.

Properties can be static as well as instance members.

A get accessor of a property has no parameters.

A set accessor of a property contains the implicit value parameter.

- class
- object
- Access Specifiers
- Fields
- Properties
- get, set
- Read only properties
- Constructors
- Destructors
- Object Initializer
- Methods
- Optional Parameters
- Named Parameters
- Positional Parameters
- Using params
- Local functions
- Project References
- Indexers



Thank You